ИТМО

Функциональная схемотехника

Язык описания аппаратуры Verilog

Преподаватель

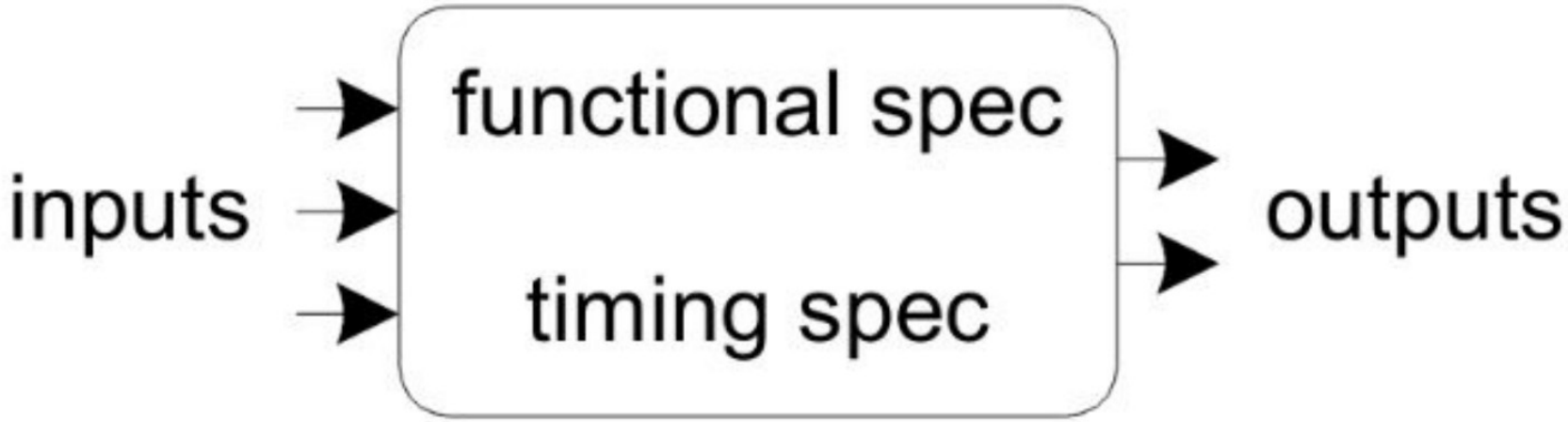
Лукашов Иван Викторович

к.т.н., старший преподаватель ПИиКТ

Санкт-Петербург, 2026

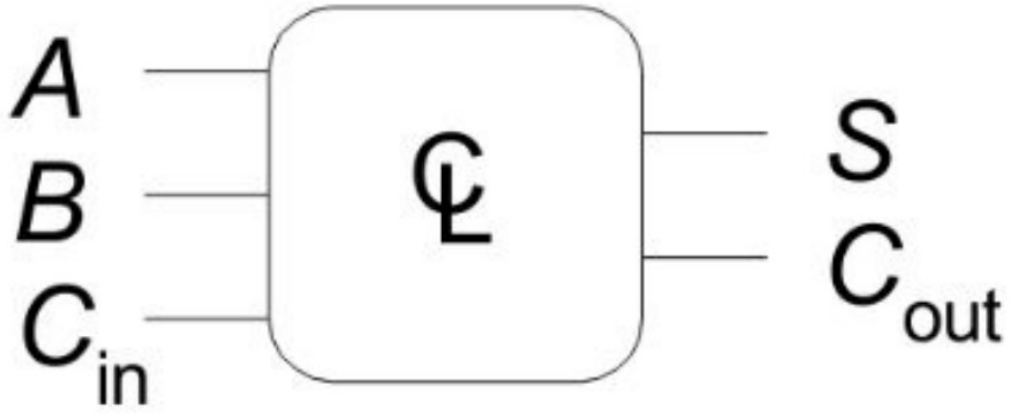
Проектируемая схема. Абстракция

- Входные порты;
- Выходные порты;
- Функциональная спецификация;
- Временная спецификация



Функциональная спецификация

- Функциональная спецификация — функция выходных значений от входных.



$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$

Синтезируемость языков описания аппаратуры (HDL)

Только некоторые конструкции HDL могут быть синтезированы в реальное аппаратное обеспечение.

Несинтезируемые конструкции используются для тестирования (testbench) и высокоуровневого моделирования.

Синтезируемые конструкции	Несинтезируемые конструкции
Базовые типы, агрегатные типы (структуры, массивы)	Сигналы с несколькими источниками (multi-driven signals)
Базовые операции	Деление, явные операции со временем (например, задержки)
Базовое управление потоком: if, циклы	Явная генерация/ожидание событий
Функции, задачи	Системные вызовы
Процедурные блоки (процессы), запускаемые изменением значения сигнала	Процедурные блоки (процессы), запускаемые началом симуляции
Инстанцирование подмодулей	Создание/завершение потоков (thread forking/joining), IPC (семафоры, почтовые ящики)

Основные типы цифровых схем

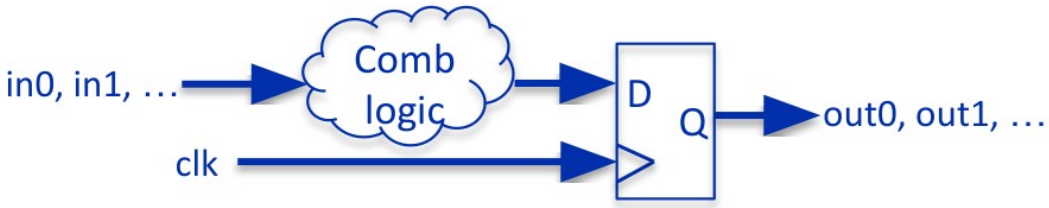
Комбинационные схемы

- не имеют внутренних состояний
- выходные значения пересчитываются сразу же после изменения входных сигналов



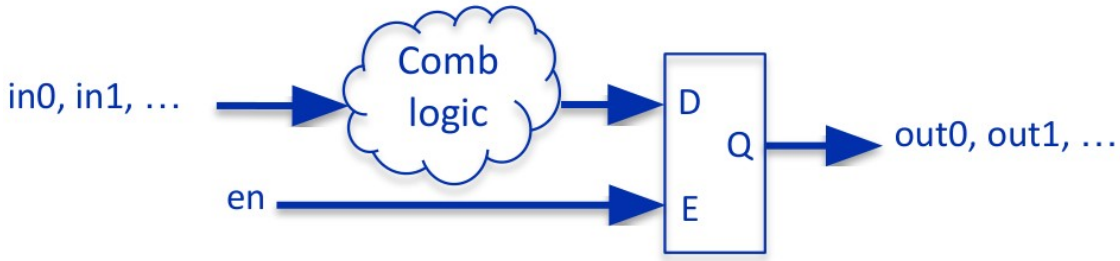
Схемы с регистрами (flip-flops)

- Содержат триггеры D-типа (D-type flip-flops)
- Выходные значения фиксируются по фронту тактового сигнала (положительному или отрицательному).



Схемы с регистрами (защёлками/latches):

- содержат защелки
- защёлка прозрачна, пока сигнал разрешения (enable) находится в высоком (или низком) состоянии, в противном случае фиксируется
- внимание: использование защёлок не рекомендуется в большинстве случаев.



SystemVerilog: структура модуля

```
module my_module_name
(
      input clk_i
      , input rst_i
      , input logic [31:0] some_src0_bi
      , input logic [31:0] some_src1_bi
      , output logic [31:0] some_result_bo
);

logic [15:0] some_internal_signal0;
logic [15:0] some_internal_signal1;

some_module some_instance0(<connections>);

always @*                              // combinational logic
      begin
      …
      end

always @(posedge clk_i)                // sequential logic
      begin
      …
      end

endmodule
```

SystemVerilog: basic types основные типы

- **Числа задаются как: WIDTH<b/h/d>VALUE, (например, 32'hdeadbeef)**

Работают с логикой с двумя состояниями (0/1) или четырьмя состояниями (0/1/x/z). Делятся на:

- **Типы цепей – wire, tri, tri0/tri1, …**

  Допустимы только непрерывные присваивания (“assign some_wire = …”)

- **Типы переменных – reg, integer, real, time, realtime, bit, byte, shortint, int, longint**

  Управляемые блоками always

- **Комбинированный тип – logic**

  Допускает оба типа присваиваний

Также доступны:

- **Перечисляемые типы: (enum), константы (const), расширения типов typedef**
- **Статические агрегированные типы: массивы, структуры (packed structure / unpacked structure), объединения (packed union / unpacked union)**
- **Динамические агрегированные типы: dynamic array, associative array, queue**

SystemVerilog: основные операции

Стандартные Си-подобные операции

- арифметические: +, -, *, /, **
- логические: !, &&, ||
- сравнения: >, <, >=, <=, ==, !=
- сдвиги: », «, »>, «<

Дополнительные операции битового уровня:

- извлечение отдельных битов: signal[msb:lsb]
- редукция: &, ~&, |, ~|, ^, ~^ ^~
- конкатенация, репликация: {, }, {n{m}}

Операции сравнения для логики с четырьмя состояниями (===, !===)

SystemVerilog: процедурные блоки

Список чувствительности (Sensitivity list) – процедурный блок выполняется, когда заданные сигналы изменяют свои значения.

Может быть вызван следующими событиями:

- **`posedge (положительный, передний, возрастающий фронт`** сигнала)
- **`negedge (отрицательный, задний, спадающий фронт сигнала)`**

Операции внутри процедурного блока выполняются по порядку.

Порядок процедурных блоков в проекте не важен (порядок выполнения не определён).

Процедурный блок не может быть прерван, пока не наступит ожидаемое событие.

```
logic cmd;
logic [31:0] in0, in1, out;

always @(cmd, in0, in1)
       begin
       if (cmd == 1'b0)
              begin
              out = in0 + in1;
              end
       else
              begin
              out = in0 * in1;
              end
       end
```

SystemVerilog: Комбинационная логика




```
always @(in0, in1)
    begin
    …
    result = in0 + in1;
    …
    end
```

```
always @*
    begin
    …
    result = in0 + in1;
    …
    end
```

```
always_comb
    begin
    …
    result = in0 + in1;
    …
    end
```

Все читаемые переменные должны быть перечислены в списке чувствительности

Все записываемые переменные должны получать новое значение (например, значение по умолчанию)

В противном случае: будет создана неявная память с уровневой чувствительностью (защёлка)

Комбинационная логика. Описание

- Оператор «assign»;
- Процедурный блок «always» и блокирующие присваивания.

```
assign result = A & B;

always @(A, B) begin
    result = A & B;
end
```

Логическое «И»

```
assign result = condition ? A : B;

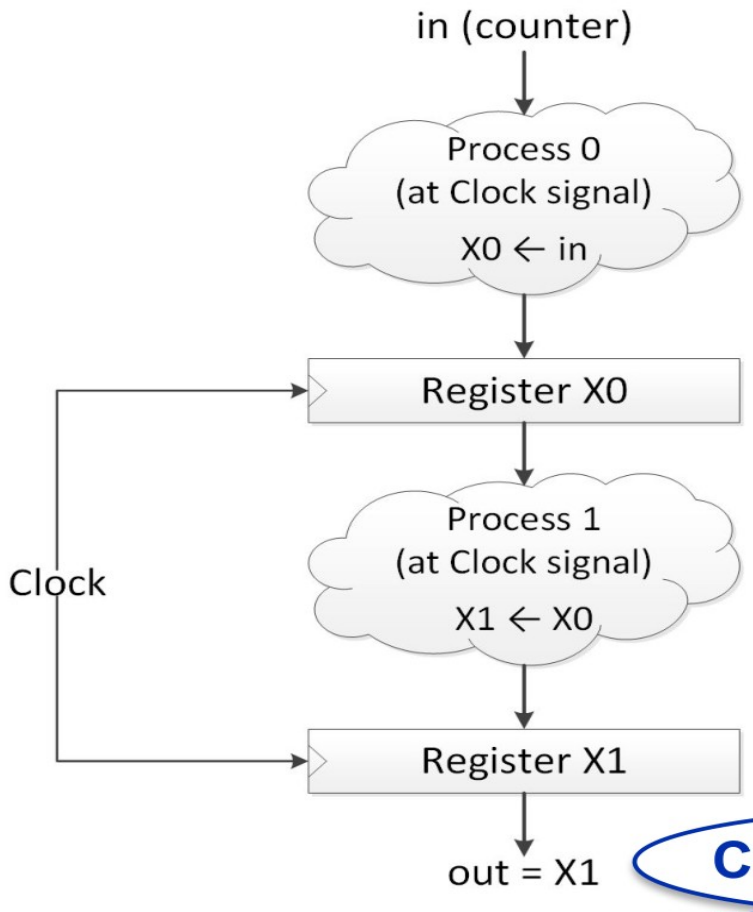
always @(condition, A, B) begin
    if (condition)
        result = A;
    else
        result = B;
end
```

Мультиплексор

```
always @(*) begin
    case (data_in)
      8'b0000_0001: data_out=3'd0;
      8'b0000_0010: data_out=3'd1;
      8'b0000_0100: data_out=3'd2;
      8'b0000_1000: data_out=3'd3;
      8'b0001_0000: data_out=3'd4;
      8'b0010_0000: data_out=3'd5;
      8'b0100_0000: data_out=3'd6;
      8'b1000_0000: data_out=3'd7;
    endcase
end
```

Шифратор

Состояние гонок при присваивании регистров? (пример 1)

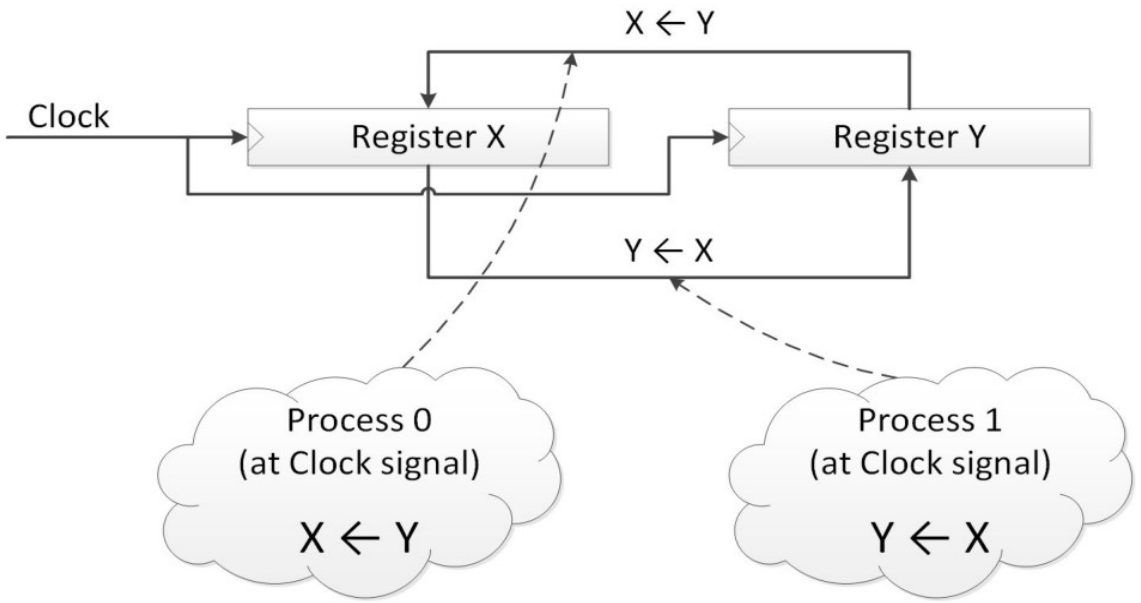


Циклы синхронизации сигнала	0 (начальное значение)	1	2	3	4	5
Ожидаемое поведение						
X0	0	1	2	3	4	5
X1	0	0	1	2	3	4
Выполнение Process 0 перед Process 1 …						
X0	0	1	2	3	4	5
X1	0	1	2	3	4	5
Выполнение Process 1 перед Process 0 …						
X0	0	1	2	3	4	5
X1	0	0	1	2	3	4

Состояние гонки!

Состояние гонок при присваивании регистров? (пример 2)

Пример: два регистра синхронно обмениваются значениями



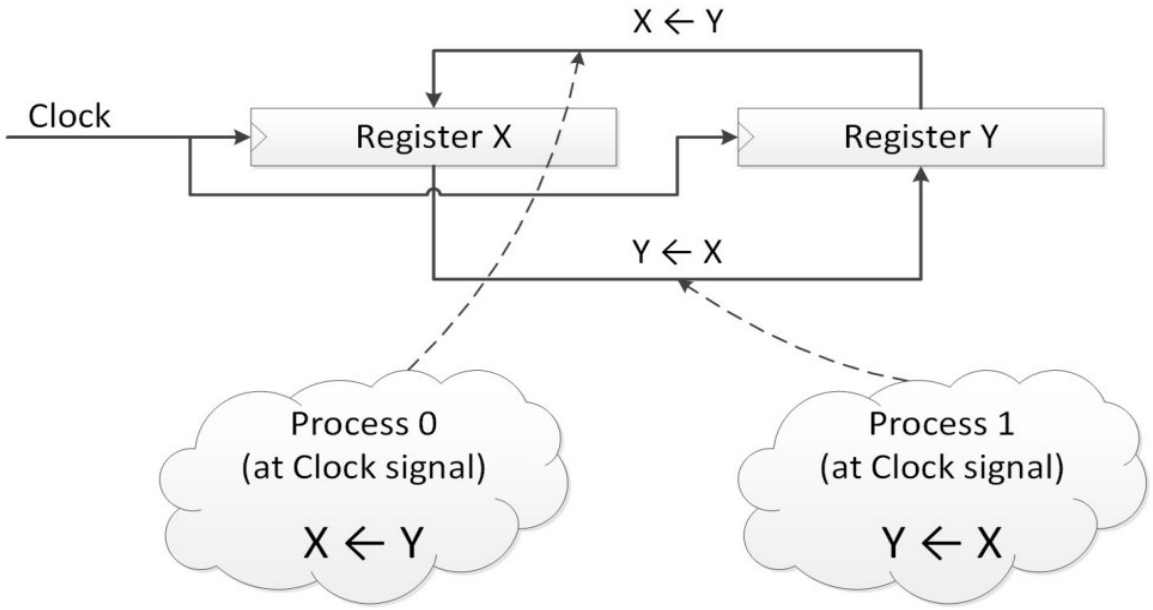
Циклы синхронизации сигнала	0 (начальное значение)	1	2	3	4	5
Ожидаемое поведение						
X	5	3	5	3	5	3
Y	3	5	3	5	3	5
Выполнение Process 0 перед Process 1 …						
X	5	3	3	3	3	3
Y	3	3	3	3	3	3
Выполнение Process 1 перед Process 0 …						
X	5	5	5	5	5	5
Y	3	5	5	5	5	5

Состояние гонки!

Решение?

Состояние гонок при присваивании регистров? (пример 2)

Пример: два регистра синхронно обмениваются значениями



Циклы синхронизации сигнала	0 (начальное значение)	1	2	3	4	5
Ожидаемое поведение						
X	5	3	5	3	5	3
Y	3	5	3	5	3	5
Выполнение Process 0 перед Process 1 …						
X	5	3	5	3	5	3
Y	3	5	3	5	3	5
Выполнение Process 1 перед Process 0 …						
X	5	3	5	3	5	3
Y	3	5	3	5	3	5

Норм

Решение: отложить присваивание до продвижения времени симуляции.

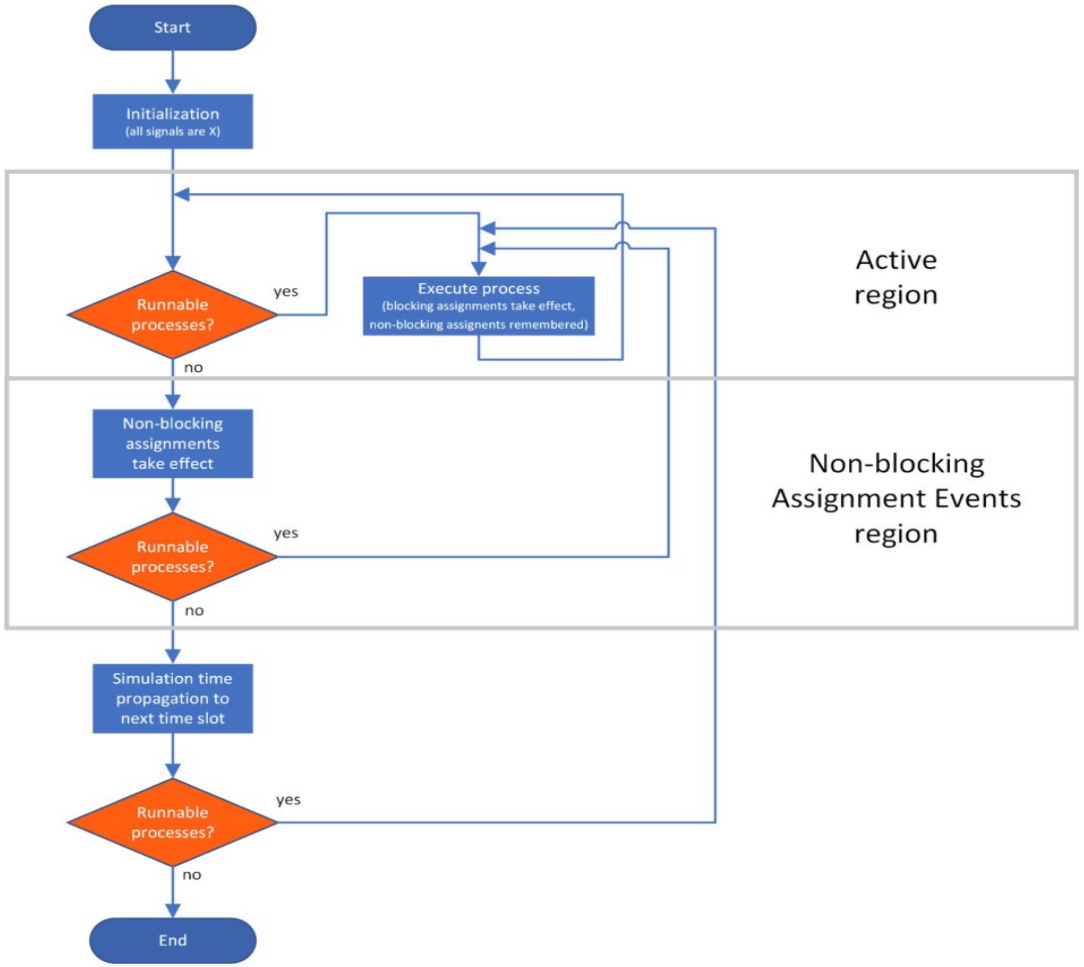
SystemVerilog: типы присваиваний

Два типа присваиваний:

- **блокирующее (=): вступает в силу немедленно**
  Используется для комбинационной логики

- **неблокирующее (<=): откладывается до следующего дельта-цикла**
  Используется для регистров

Ядро симуляции

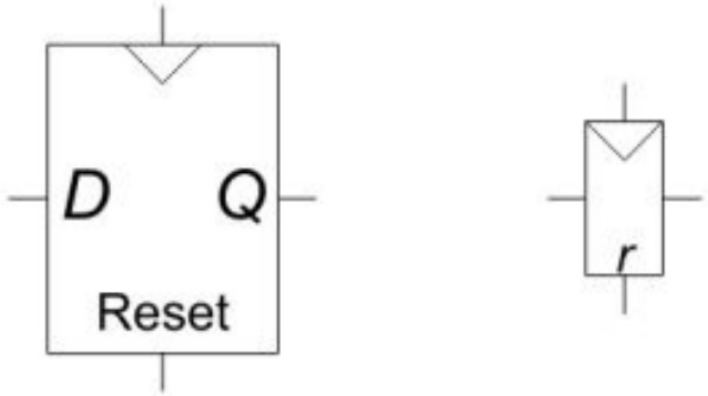


- **Симулятор итеративно обрабатывает** упорядоченные по времени списки событий и процессы, реагирующие на них.

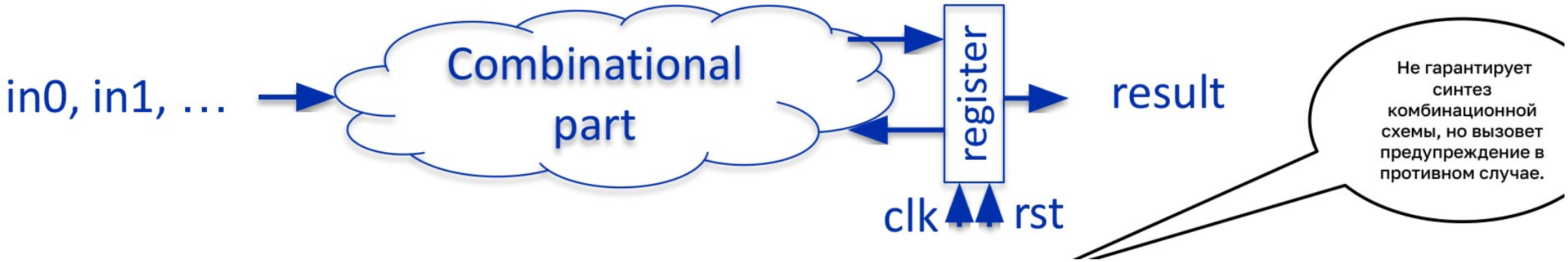
- Блокирующие и неблокирующие присваивания **вступают в силу на разных фазах симуляции,** чтобы избежать условий гонки.

Последовательностная логика

- Выходы схемы зависят от текущих и от предыдущих входных значений;
- Схемы с «памятью»;
- Предыдущие значения схемы формируют текущее «состояние» схемы;
- Текущее «состояние» схемы влияет на её будущее состояние;
- Последовательность входных сигналов имеет значение;



SystemVerilog: последовательностная логика




```
always @(posedge clk)
      begin
      …
      if (rst)
            result <= 0;
      else
            result <= in0 + in1;
      …
      end
```

```
always_ff @(posedge clk)
      begin
      …
      if (rst)
            result <= 0;
      else
            result <= in0 + in1;
      …
      end
```

Сигналы могут получать новые значения по условиям.

Последовательностная логика. Описание

- Always-блок. Реакция на фронт/спад тактового сигнала

```
always @(posedge clk) begin
    op1_ff <= op1_next;
    op2_ff <= op2_next;
end
```

```
always @(posedge clk) begin
    if (enable) begin
        op1_ff <= op1_next;
        op2_ff <= op2_next;
    end
end
```

```
always @(posedge clk) begin
    if (~rst_n) begin
        op1_ff <= 8'b00000000;
        op2_ff <= 8'b00000000;
    end else if (enable) begin
        op1_ff <= op1_next;
        op2_ff <= op2_next;
    end
end
```

Good coding style

- Не смешивать описания комбинационной и последовательностной логики в одном always-блоке;

```
always @(posedge clk) begin
    op1_ff <= A + B;
    op2_ff <= condition ? A : ~A + 1;
end
```

Некорректно

```
wire op1_next;
wire op2_next;

assign op1_next = A + B;
assign op2_next = condition ? A : ~A + 1;

always @(posedge clk) begin
    op1_ff <= op1_next;
    op2_ff <= op2_next;
end
```

Корректно

Good coding style

- Не смешивать описания комбинационной и последовательностной логики в одном always-блоке;

```
always @(posedge clk) begin
    op1_ff <= A + B;
    op2_ff <= condition ? A : ~A + 1;
end
```

Некорректно

```
wire op1_next;
wire op2_next;

assign op1_next = A + B;
assign op2_next = condition ? A : ~A + 1;

always @(posedge clk) begin
    op1_ff <= op1_next;
    op2_ff <= op2_next;
end
```

Корректно

Good coding style

- Блокирующие присваивания для комбинационной логики;
- Неблокирующие присваивания для последовательностной логики.

```
always @(posedge clk) begin
    op1_ff = op1_next;
    op2_ff = op2_next;
end

always @(condition, A, B) begin
    if (condition)
        result <= A;
    else
        result <= B;
end
```

Некорректно

```
always @(posedge clk) begin
    op1_ff <= op1_next;
    op2_ff <= op2_next;
end

always @(condition, A, B) begin
    if (condition)
        result = A;
    else
        result = B;
end
```

Корректно

Good coding style

- Не смешивать блокирующие и неблокирующие присваивания в рамках одного блока.

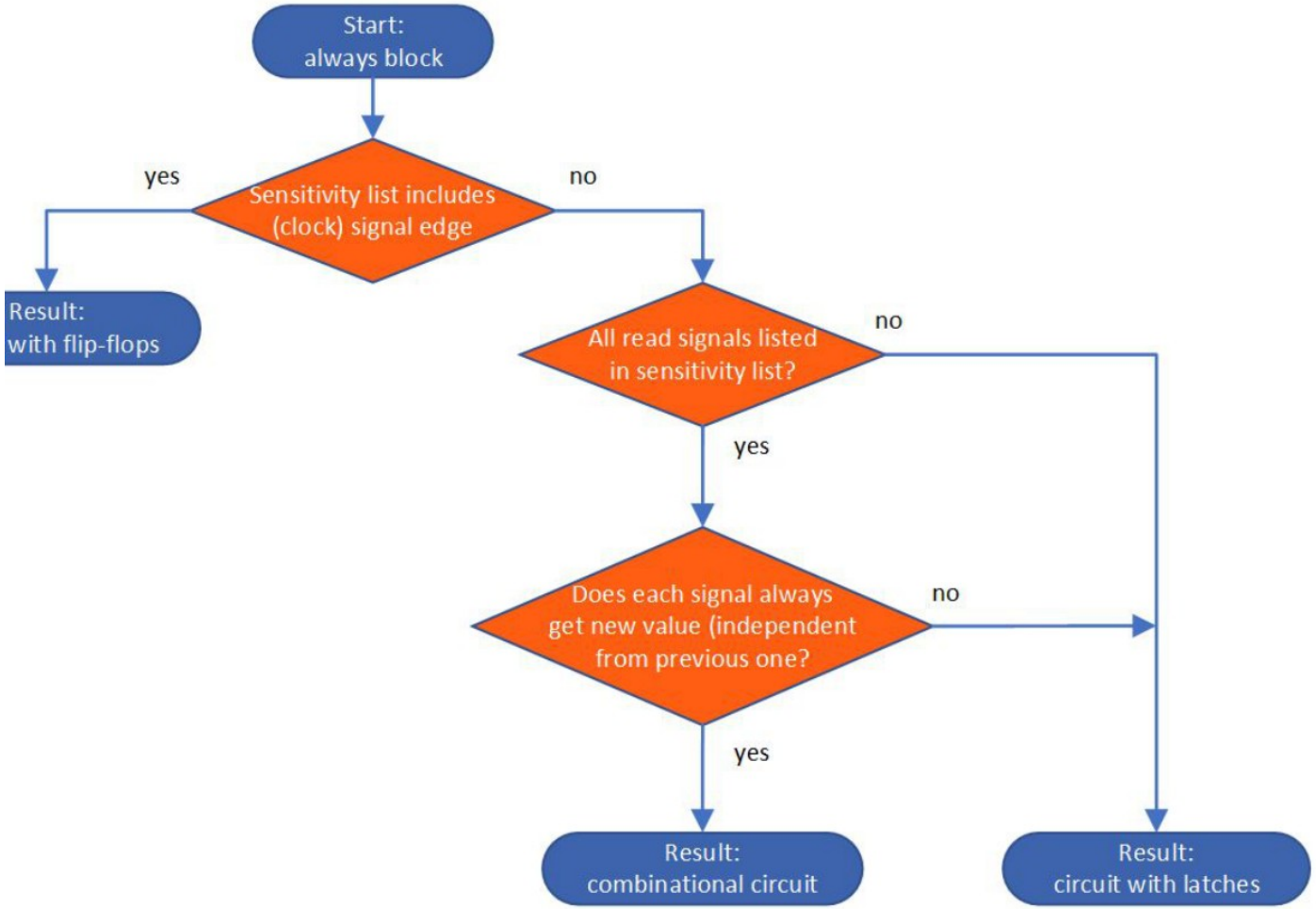
```
always @(posedge clk) begin
    op_new_ff = opnew_next;
    op_old_ff <= op_new_ff;
end
```

Некорректно

```
always @(posedge clk) begin
    op_new_ff <= opnew_next;
    op_old_ff <= op_new_ff;
end
```

Корректно

Классификация схем по используемым always блокам (для большинства случаев)



Пример: сдвиговый регистр на SystemVerilog (2 always блока)

```
module shifter (
    input clk_i
    , input rst_i
    , input load_cmd_i
    , input [31:0] load_data_bi
    , input rshift_cmd_i
    , output logic [31:0] dataout_bo
);

logic [31:0] shift_register, shift_register_next;

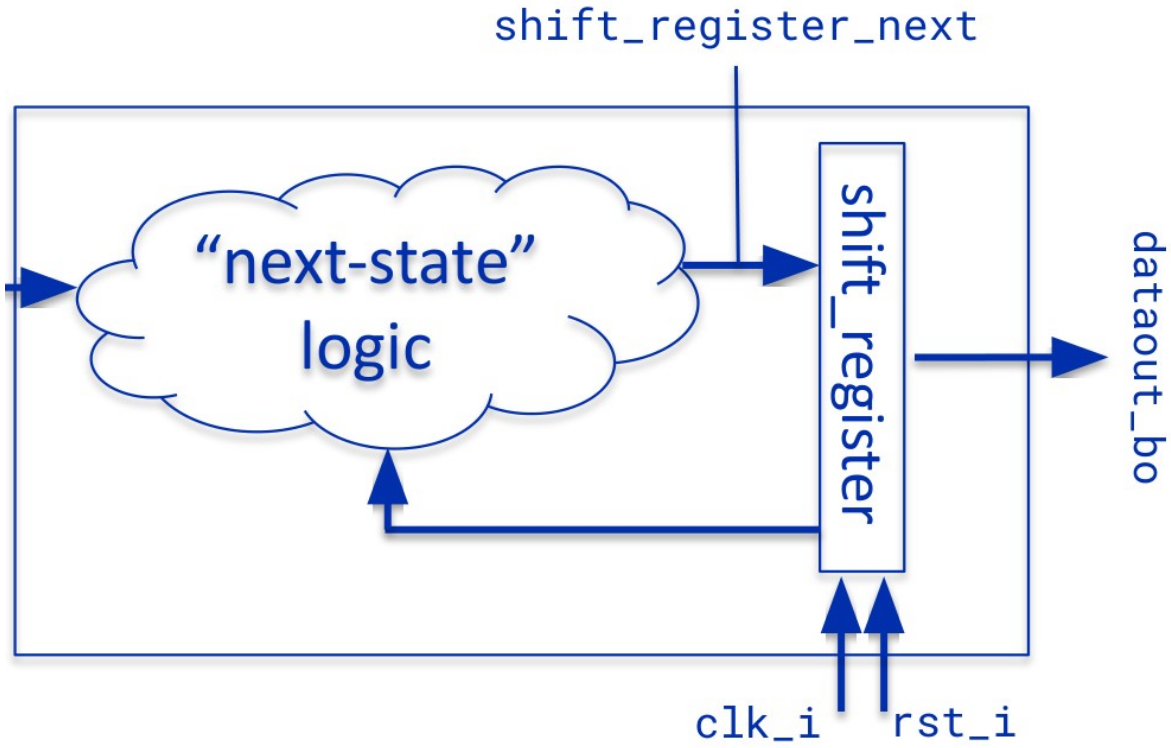
// combinational logic
always @*
    begin
    shift_register_next = shift_register;
    if (load_cmd_i) shift_register_next = load_data_bi;
    else if (rshift_cmd_i) shift_register_next = shift_register_next >> 1;
    end

// state register
always @(posedge clk_i)
    begin
    if (rst_i) shift_register <= 32'h0;
    else shift_register <= shift_register_next;
    end

assign dataout_bo = shift_register;

endmodule
```

```
load_cmd_i,
load_data_bi,
rshift_cmd_i
```



Присвоения SystemVerilog

Тип присваиваний	Сложные always-блоки	Присваивания без гонок к регистрам	Объём кода (увеличивается)
Только блокирующие	Да	Нет	Нет
Только неблокирующие	Скорее, нет, промежуточные присваивания не видны сразу	Да	Нет
Оба типа: блокирующие для комбинационной логики и неблокирующие для регистров	Да	Да	Да, 2x always-блоки, переменные *_next и значения по умолчанию

Присвоения SystemVerilog

Тип присваиваний	Сложные always-блоки	Присваивания без гонок к регистрам	Объём кода (увеличивается)
Только блокирующие	Да	Нет	Нет
Только неблокирующие	Скорее, нет, промежуточные присваивания не видны сразу	Да	Нет
Оба типа: блокирующие для комбинационной логики и неблокирующие для регистров	Да	Да	Да, 2x always-блоки, переменные *_next и значения по умолчанию

SystemVerilog: Иерархическое проектирование

```
logic clk, rst;
logic [31:0] src0, src1, result;

submodule_type
#(
      .param0(some_val0)
      , .param1(some_val1)
)
submodule_name
(
      .clk_i(clk)
      , .rst_i(rst)
      , .datain0_bi(src0)
      , .datain1_bi(src1)
      , .dataout_bo(result)
);
```

SystemVerilog: объявление интерфейсов

```
interface MemSplit32 ();
    logic req;
    logic ack;
    logic [31:0] addr;
    logic we;
    logic [31:0] wdata;
    logic [3:0] be;
    logic resp;
    logic [31:0] rdata;

    modport Master
       (output req, input ack, output addr, output we, output wdata, output be, input resp, input rdata);

    modport Slave
       (input req, output ack, input addr, input we, input wdata, input be, output resp, output rdata);

    modport Monitor
       (input req, input ack, input addr, input we, input wdata, input be, input resp, input rdata);

 endinterface
```

SystemVerilog: использование интерфейсов

```
module pipeline
(
    input clk_i
    , input rst_i

    , MemSplit32.Slave datain
    , MemSplit32.Master dataout
);

MemSplit32 data_inter();

stage_0 stage_0_inst
          (.clk_i(clk_i), .rst_i(rst_i), .datain(datain), .dataout(data_inter));

stage_1 stage_1_inst
          (.clk_i(clk_i), .rst_i(rst_i), .datain(data_inter), .dataout(dataout));

monitor monitor_inst
          (.clk_i(clk_i), .rst_i(rst_i), .datain(data_inter), ...);

endmodule
```

